

The Ultimate Python 100+ Question Bank (Easy English Version)

Module 1: Syntax, Variables & Conditional Logic (1–10)

1. Swap the values of two variables $a = 15$ and $b = 25$ without using a third variable.
2. Take a year as input from the user and check if it is a Leap Year.
3. Take an integer input and check if it is Even or Odd using a conditional operator in a single line of code.
4. Take 3 numbers as input from the user and find the largest number among them.
5. Take a temperature input in Celsius and convert it into Fahrenheit.
6. Check a user's age and NID status to determine if they are eligible to vote.
7. Take a character input and check whether it is a Vowel or a Consonant.
8. Create a number grading system ($80-100 = A+$, $70-79 = A$, etc.).
9. Take two numbers and an operator (+, -, *, /) as input and build a simple calculator.
10. Check if a number is positive, negative, or zero using nested if-else statements.

Module 2: Loop Mechanics & Control Flow (11–20)

11. Find the sum of all even numbers between 1 and 50 using a for loop.
12. Find the factorial of a number using a while loop.
13. Keep taking inputs from the user until they enter 0, then display the average of all entered numbers.
14. Print the first 10 numbers of the Fibonacci Series using a loop.
15. Use break inside a loop to find the first number between 1 and 20 that is divisible by 7, print it, and stop the loop.
16. Loop through numbers from 1 to 15, but skip multiples of 3 and 5 using continue.
17. Create a right-angled triangle star (*) pattern using nested loops.
18. Take a number as input and check if it is a Prime Number using a loop.
19. Reverse a given input number using a loop (Example: $123 \rightarrow 321$).
20. Implement the "FizzBuzz" challenge for all numbers from 1 to 100.

Module 3: Data Structures - Lists & Tuples (21–35)

21. Remove all duplicate values from a list manually using a loop.
22. Find the largest and smallest numbers in a list without using any built-in functions.
23. Reverse a list completely without using any built-in methods like .reverse().
24. Merge two separate lists into a single list without keeping any duplicate items.
25. Create a new list containing the lengths of all string elements from an existing list.
26. Use list comprehension to create a list of squares of numbers between 1 and 30 that are divisible by 3.
27. Write the logic to rotate the elements of a list by a specific distance (e.g., 2 positions to the right).
28. Find the index number of a specific element inside a tuple.
29. Calculate the sum of the second elements of each tuple from a list of tuples: [(1, 2), (3, 4), (5, 6)].
30. Separate only the float numbers from a mixed data type list and put them into a new list.
31. Find the common elements between two lists and save them into a new list.
32. Manually count the frequency of each element in a list (how many times each item appears).
33. Flatten a nested list [1, [2, 3], 4, [5, 6]] into a single, flat list.

34. Take 5 names as input from the user and sort them in alphabetical order (A-Z).
35. Receive multiple return values from a function using tuple unpacking.

Module 4: Data Structures - Dictionaries & Sets **(36–50)**

36. Show 3 different ways in code to merge two separate dictionaries together.
37. Calculate the sum of all values inside a dictionary.
38. Delete a specific key and its value completely from a dictionary.
39. Use dictionary comprehension to create a dictionary of cubes of numbers from 1 to 10 (x: x³).
40. Count how many times each character appears in a string using dictionary mapping.
41. Invert a dictionary by swapping its keys and values with each other.
42. Access a deep-level nested value inside a dictionary safely using the .get() method.
43. Sort a dictionary based on its values instead of its keys.
44. Perform union, intersection, and difference operations between two sets.
45. Demonstrate how to use a set to easily remove duplicates from a list.
46. Remove a specific element from a set in a way that the code does not crash even if the item does not exist.
47. Check if a specific key exists in a dictionary using conditional logic.
48. Create a frozen set and write code to prove that it cannot be modified (immutable).
49. Collect all unique values from a dictionary and store them inside a set.
50. Find the common keys between two dictionaries using the set intersection method.

Module 5: Functions, Scope & Modular Code Design (51–60)

51. Create a function that accepts an unlimited number of arguments (*args) and returns their product (multiplication).
52. Write a function to print user profile data using keyword arguments (kwargs).
53. Create a function to find the factorial of a number using recursion.
54. Show how to modify a global variable from inside a local function scope.
55. Create a function that takes another function as an argument (Higher-Order Function).
56. Build a reusable function to check if a string is a Palindrome (reads the same backward as forward).
57. Show the complete process of creating a custom Python module and importing it into another file.
58. Find the square root of a number using a lambda function.
59. Design a function that utilizes default parameters.
60. Use type hinting to specify and fix the input and output data types of a function.

Module 6: Object-Oriented Programming (OOP) - Core Concepts (61–70)

61. Create a Student class with attributes name and roll. Create two separate objects from this class.
62. Initialize instance variables of an object using the constructor method (__init__) inside a class.
63. Create a Rectangle class with custom methods to calculate its area and perimeter.
64. Make a class variable private using encapsulation rules and create getter and setter methods for it.
65. Create a child class Dog from a parent class Animal using single inheritance.

66. Change the logic of a parent class method inside a child class using method overriding.
67. Show a professional, real-world example of multiple inheritance in code.
68. Call the parent class constructor from a child class using the `super()` keyword.
69. Use a loop on different objects containing the same method name to demonstrate polymorphism.
70. Write a code example that clearly shows the difference between a class variable (static) and an instance variable.

Module 7: Advanced OOP & Dunder Methods (71–80)

71. Implement the `__str__` magic method so that printing a custom object displays a clean, readable text message.
72. Override the `__add__` method to directly add two custom objects together (`obj1 + obj2`).
73. Use magic methods like `__eq__` or `__lt__` to compare the age or value of two separate objects.
74. Implement the `__call__` method to make a custom class object callable like a standard function (`obj()`).
75. Write a code example that shows the actual usage and difference between `@classmethod` and `@staticmethod` decorators.
76. Customize the `__len__` magic method to allow the built-in `len()` function to work on your custom object.
77. Create a read-only attribute using the `@property` decorator.
78. Show how to check the Method Resolution Order (MRO) of a class in Python.
79. Demonstrate how to create an abstract class and define abstract methods.
80. Use the `__getitem__` method to allow a custom object to be accessed using keys, just like a dictionary.

Module 8: Persistent Storage & Exception Handling (81–90)

1. Create a text file, write 3 lines of data into it, and then close the file properly.
2. Write code to read a file safely without risking memory leaks by using a `with open ()` block.
3. Show how to open a file in append mode to add a new line at the bottom without erasing old text.
4. Demonstrate how to convert a Python dictionary into a JSON file using `Json. Dump()`.
5. Load data from a JSON file and convert it back into a standard Python dictionary.
6. Handle a division by zero error (`ZeroDivisionError`) using a `try-except` block.
7. Use multiple specific `except` blocks for a single `try` block to build a robust error defense.
8. Create a loop that handles `Value Error` when a user enters invalid input, prompting them to try again until correct.
9. Use a `final` block in file operations to ensure a file closes properly whether an error occurs or not.
10. Manually trigger or throw an error inside your code based on a condition using the `raise` keyword.

Module 9: Functional Tools & Memory Optimization (91–100)

91. Double all the numbers in a list using a combination of the `map()` function and a `lambda` function.

91. Filter out all the odd numbers from a list using the `filter()` function.
92. Find the product (multiplication result) of all numbers in a list in a single line using `functools.reduce()`.
93. Print the index and the corresponding element of a list simultaneously using `enumerate()` inside a loop.
94. Use the `zip()` function to pair two different lists together and loop through them at the same time.
95. Create a memory-saving sequence generator using a custom generator function and the `yield` keyword.
96. Show the effectiveness of a generator expression in processing millions of data points without crashing system memory.
97. Create a custom decorator that automatically prints a message right before and right after the main function executes.
98. Search for all phone numbers or email patterns inside a large text block using the `re` module (Regex).
99. Use `copy.deepcopy()` to avoid memory reference bugs when copying nested objects or lists.

Module 10: Pro-Max Enterprise Level Challenges **(101–105)**

1. Run 3 asynchronous tasks concurrently (at the same time) without blocking code execution using the `asyncio` library.
2. Perform heavy mathematical calculations by running multiple tasks across different CPU cores using multiprocessing.
3. Create a dictionary using `collections.defaultdict` that automatically initializes an empty list for missing keys instead of crashing with a `KeyError`.
4. Set up a professional, modular project directory structure including custom type hinting, custom exceptions, and file logging configuration.
5. Design 3 automated test cases for a custom math function using the built-in `unittest` module inside an object-oriented testing environment.